

also saves details for the model graphs, loss functions and a special index that tracks how close the images match each other and how much they resemble the object being identified regarding percentages.

Building your own model

```

• import tensorflow as tf
• # Assuming object detection API is available for use
• from object_detection.utils.config_util import create_
pipeline_proto_from_configs
• from object_detection.utils.config_util import get_con-
figs_from_pipeline_file
• import object_detection.exporter
• # Configuration for model to be exported
• config_pathname = ${Model_configuration}
• # Input checkpoint for the model to be exported
• # Path to the directory which consists of the saved model
on disk (see above)
• trained_model_dir = ${Model_dir_path}
• # Create proto from model configuration
• configs = get_configs_from_pipeline_file(config_path-
name)
• pipeline_proto = create_pipeline_proto_from_
configs(configs=configs)
• # Read .ckpt and .meta files from model directory
• checkpoint = tf.train.get_checkpoint_state(trained_
model_dir)
• input_checkpoint = checkpoint.model_checkpoint_path
• # Model Version
• model_version_id = ${Enter_model_version}
• # Output Directory
• output_directory = ${Output_directory} + str(model_
version_id)
• # Export model for serving
• object_detection.exporter.export_infer-
ence_graph(input_type='image_tensor',pipeline_
config=pipeline_proto,trained_checkpoint_prefix=input_
checkpoint,output_directory=output_directory)

```

The above code is to be used for building the model in the case that there may be some debugging required before compiling it. Here's what it all means.